

AutoRecover

An AI payment-recovery engine that decides **whether**, **when** and **on which rail** to retry a failed payment — and never lets a language model authorise money movement.

₹11.71L recovered per ₹1Cr of failed volume

1,016 tests · CI green

Built for Bharat · RBI e-mandate compliant

THE PROBLEM

A failed payment is not a lost customer. It is money still on the table — and almost nobody goes back for it properly.

19.2%

is all a fixed 3-retry schedule recovers — the industry default. Calibrated to the 15–30% band public figures report.

₹81.5L

walks away per ₹1 crore of failed volume under that default. UPI timeouts, bank downtime, expired mandates — most of it temporary, all of it recoverable.

8.0%

of its attempts are false retries — measured. Gateway cost spent, and customers nudged for money they had already paid.

The dumb half: sending

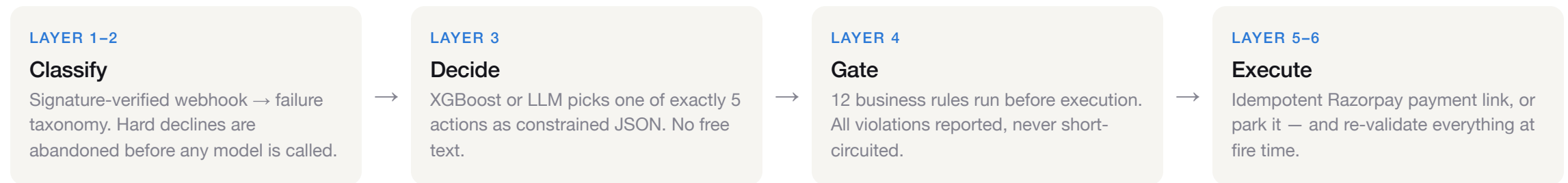
A fixed schedule knows nothing about *why* the payment failed, which bank is down right now, or whether this customer already said stop.

The ignored half: listening

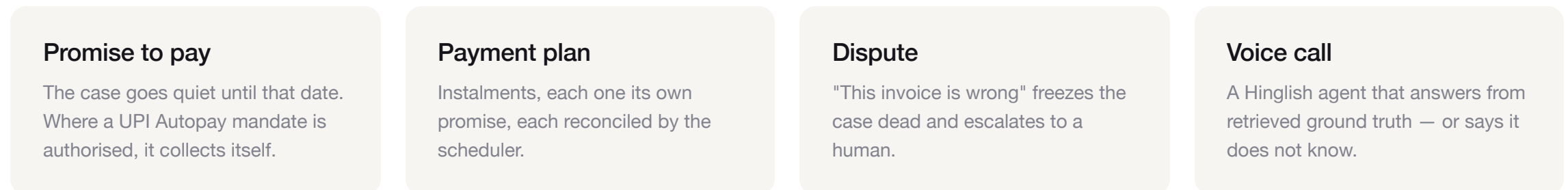
Manual dunning does not scale, and it breaks the rules — calls at 11pm, four messages to one B2B buyer, chasing an invoice the customer disputes.

THE SOLUTION

One pipeline. Deterministic at both ends, intelligent in the middle.



Sending the link is the easy half. The engine also listens to the reply.



Eight layers. The model sits in the middle, wrapped by deterministic code on both sides.

LAYER 1	Ingestion — HMAC signature verify → idempotency → event store. Razorpay webhooks and merchant-pushed risks
LAYER 2	Classifier — error code → failure taxonomy. Retryable, hard-decline, switch-only. No LLM here
LAYER 3	Policy agent — XGBoost or LLM → constrained JSON: retry_now, retry_at, switch_rail, nudge_customer, abandon
LAYER 4	Guardrail gate — 12 business rules + schema validation, before execution, all violations reported
LAYER 5	Messaging — scoped LLM generation for the customer nudge. English and Hindi recovery page
LAYER 6	Scheduler — 14 sweeps per tick: fire deferrals, reconcile dropped events, expire promises, chase due cases
LAYER 7	Receivables — B2B dunning ladder by role, aging, disputes, payment plans, consolidated statements
LAYER 8	Voice — Hinglish call handling, 4 gates, promise capture from speech

The four chasers extend the same pipeline to money that never reached a gateway — abandoned carts, halted subscriptions, overdue invoices, failed mandate debits. The merchant POSTs them to `/risks` and they run through the identical agent → guardrail → payment-link machinery.

RESULTS

₹11.71 lakh more net revenue per ₹1 crore of failed volume — at 15% fewer retry attempts.

POLICY	RECOVERY RATE	ATTEMPTS (AVG)	FALSE RETRIES	NET ₹ PER ₹1CR
No retry	0.0%	0.00	0.0%	₹0
Fixed 3-retry (baseline)	19.2% ± 0.7	2.73	8.0%	₹17,95,744
AutoRecover · XGBoost	30.2% ± 0.5	2.32	0.0%	₹29,66,249
AutoRecover · LLM agent	26.6% ± 0.5	2.38	0.0%	₹25,97,603

Recovery rate, 5,000 scenarios × 5 seeds



+11.03pp, and it is real
95% CI [+10.54, +11.52] over n = 25,000, under common random numbers — every policy sees the identical scenario draw, and outcomes are differenced one-to-one. Overlapping ± ranges cannot answer this question, so the harness does not use them.

Four things we tell you before you ask.

The bank simulator is synthetic

Its central assumption: a retry is not a fresh payment. What matters is $P(\text{the blocker cleared})$, modelled per failure class. Blended baseline lands at 19.2% — inside the 15–30% band public figures report, and a calibration test fails the build if a change pushes it out.

The LLM lost

26.6% against XGBoost's 30.2%, on identical scenarios — and the ₹ figures do not even charge it for inference. Nothing in this eval justifies the LLM for this task. It stays in the harness as the interesting negative result.

Net, not gross

Recovery rate alone makes brute force optimal by construction. The headline is net of retry cost at ₹2.00/attempt — and because AutoRecover recovers more while attempting fewer, it dominates the baseline at any retry cost, including ₹0. The assumption never has to be argued.

A dead provider cannot fake a row

The policy agent swallows LLM errors and falls back, so a broken key would otherwise produce a plausible LLM row made entirely of XGBoost decisions. The runner counts fallbacks and **drops the row** when they dominate. An earlier README shipped with it blank for exactly that reason.

What stops it doing something stupid with real money.

- **No LLM ever authorises money movement.** It selects one of 5 fixed actions; deterministic code executes.
- **Hard declines never reach the model.** Fraud blocks and stolen cards are abandoned at Layer 2.
- **12 business rules, all checked.** Max 3 retries per payment, 5 per customer per rolling 24h, ₹50K ceiling, 72h consent window, 2 nudges/day.
- **No short-circuiting.** Every rule runs and every violation is recorded, because the audit log is the product.
- **Idempotency is enforced before the API call.** A deterministic key with a UNIQUE constraint — a replayed webhook cannot create a second payment link.
- **Deferred retries re-validate at fire time.** A customer who opts out at 23:00 does not get the 02:00 retry approved at 22:00.
- **Write-ahead execution.** Every attempt is committed pending before Razorpay is called, so a process death mid-call leaves money findable, not lost.
- **An open dispute freezes the case.** Nothing automated touches it until a human resolves it.

Deterministic logic wraps the model on **both** sides. That is the entire architectural claim, and it is the first thing a fintech panel checks.

Indian payments are not a localisation layer. They are the design.

UPI Autopay, inside RBI's e-mandate rules

A promise backed by a mandate collects itself on the date the customer named. The guardrail enforces the $\geq 24\text{h}$ pre-debit notice — no notice, and the debit is *turned into* the notice rather than attempted. Above the ₹15,000 authentication-exempt ceiling the option is never offered, because an unattended debit is not lawful there.

A true Asia/Kolkata clock

The 11pm–7am contact blackout is computed on a real IST clock, not a server offset. A deferral that would land inside the window is shifted forward to its edge before it is parked — forward-only, because waiting longer is always compliant.

Hinglish voice, four gates

Opt-out honoured first, a retrieval floor below which it abstains, sanitation of instructions hiding in retrieved text, and a grounding check the answer must pass against its cited passage. A confident invented number on a call about money is worse than no answer.

Razorpay-native, Hindi recovery page

Test-mode webhooks and Payment Links end to end. The customer sees what went wrong in their own words, then every way out: pay, promise a date, ask for instalments, flag a problem, or stop hearing from us — in English or Hindi.

Not a prototype that demos. A system that is tested where money moves.

1,016

tests across 59 files

80%

statement coverage over src/

100%

on the idempotency guard, the taxonomy and the B2B ladder

98%

on the guardrail gate and its 12 rules

The tests that exist because money can move without a customer present

- A debit without the RBI pre-debit notice is refused
- A notice younger than 24h is not enough
- A second sweep never charges twice
- A timeout after the charge still leaves the money findable

And the failures are documented, not hidden

The README carries a standing "documented failure cases" section and a hardening log — including the release where a loaded XGBoost model was never actually consulted, and the batch that approved abandons where only CI could see it. Mutation testing runs in its own workflow.

CI green on main

Gitleaks + Dependabot

Running today, with the honest boundary marked.

Working now

- Full pipeline end to end against Razorpay test mode
- Operations console, model page, merchant console, customer recovery page
- Eval harness reproducible from a single command
- One-command local demo — no account, no Docker, no internet
- Deployable to Render: web services and Postgres, config in-repo

Next

- Live pilot against real merchant volume — the simulator is a model, not a measurement
- UPI Autopay collection switched on once Razorpay Recurring is enabled on the account
- Replace the in-process scheduler with a real queue past one app process
- An Android merchant app over the same API — the console is already server-rendered and mobile-first

CODE, DOCS AND THE EVAL HARNESS

github.com/jitheender-ops/iqoo-autorecover

Jitheender Kumar Manapuram
iQOO Hackathon 2026 · FinTech and Commerce